

scripts/06_robustness_study.py

202 lines

```
1 """
2 06_robustness_study.py
3 =====
4 Monte-Carlo robustness study.
5
6 Passing the safety audit on ONE random seed proves very little - it could be
7 luck. This script re-runs all three scenarios across many independent sensor-
8 noise realisations and asserts that the operating envelope is respected in
9 EVERY run, reporting the worst-case margin actually observed.
10
11 It also answers the question a reviewer should ask about the safety margins:
12 "how much production are they costing you?" The study reports the achieved
13 rate against the ground-truth maximum safe rate computed directly from the
14 plant's own steady-state curve.
15
16 Outputs
17 -----
18 data/robustness_summary.json
19 figures/robustness.png (margin distributions + achieved-rate spread)
20 """
21
22 from __future__ import annotations
23
24 import json
25 import sys
26 from pathlib import Path
27
28 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
29
30 import numpy as np
31 import pandas as pd
32 import matplotlib.pyplot as plt
33
34 from config import DATA_DIR, FIG_DIR, LIMITS, CONTROLLER, SCENARIOS, TS_HOURS
35 from well_simulator import WellSimulator
36 from model import WellModel
37 from controller import ChokeMPC
38 from plotting import COLOR_WHP, COLOR_FLP, COLOR_BHP, COLOR_Q, style_axis, savefig
39
40 N_RUNS = 100 # independent noise realisations per scenario
41
42
43 def target_at(targets, t):
44     val = targets[0][1]
45     for start_h, v in targets:
46         if t >= start_h:
47             val = v
48     return val
49
50
51 def run_once(key: str, model: WellModel, seed: int) -> dict:
52     """One closed-loop run of a scenario with a given noise seed."""
53     spec = SCENARIOS[key]
54     sim = WellSimulator(seed=seed, noise=True)
55     sim.reset()
56
57     controller = ChokeMPC(model=model)
58
59     u = spec["u_initial"]
60     meas = {
61         "Q": 0.0,
62         "WHP": sim.history[-1]["WHP"],
63         "FLP": sim.history[-1]["FLP"],
64         "BHP": sim.history[-1]["BHP"],
65     }
66
67     rows = []
68     for k in range(int(spec["duration_h"] / TS_HOURS)):
```

```

68 t = k * TS_HOURS
69 target = target_at(spec["targets"], t)
70 u = controller.step(meas, u, target).u_selected
71 Q, WHP, FLP, BHP = sim.step(u)
72 meas = {"Q": Q, "WHP": WHP, "FLP": FLP, "BHP": BHP}
73 rows.append({"t": t + TS_HOURS, "target": target, "u": u,
74 "Q": Q, "WHP": WHP, "FLP": FLP, "BHP": BHP})
75
76 df = pd.DataFrame(rows)
77 L = LIMITS
78 # Signed distance to the nearest limit (negative == violated)
79 margins = {
80 "WHP": float(np.minimum(df["WHP"] - L.whp_min, L.whp_max - df["WHP"]).min()),
81 "FLP": float(np.minimum(df["FLP"] - L.flp_min, L.flp_max - df["FLP"]).min()),
82 "BHP": float(np.minimum(df["BHP"] - L.bhp_min, L.bhp_max - df["BHP"]).min()),
83 }
84 max_move = float(df["u"].diff().abs().max())
85 tail = df.tail(20)
86
87 return {
88 "seed": seed,
89 "margins": margins,
90 "worst_margin": min(margins.values()),
91 "violated": min(margins.values()) < 0,
92 "max_move": max_move,
93 "ramp_ok": max_move <= CONTROLLER.max_move + 1e-6,
94 "final_Q": float(tail["Q"].mean()),
95 "final_target": float(df["target"].iloc[-1]),
96 "choke_std_tail": float(tail["u"].std()),
97 "has_nan": bool(df.isna().any().any()),
98 }
99
100
101 def main():
102 model = WellModel.load(DATA_DIR / "fitted_model.json")
103
104 # Ground truth: best steady rate that keeps every pressure inside limits.
105 q_max_safe, u_max_safe = WellSimulator(noise=False).max_safe_rate(LIMITS)
106
107 results = {}
108 all_pass = True
109
110 for key in ("A", "B", "C"):
111 runs = [run_once(key, model, seed=1000 + i) for i in range(N_RUNS)]
112 n_viol = sum(r["violated"] for r in runs)
113 n_ramp_bad = sum(not r["ramp_ok"] for r in runs)
114 n_nan = sum(r["has_nan"] for r in runs)
115 worst = min(r["worst_margin"] for r in runs)
116 finals = np.array([r["final_Q"] for r in runs])
117 ok = (n_viol == 0) and (n_ramp_bad == 0) and (n_nan == 0)
118 all_pass = all_pass and ok
119
120 results[key] = {
121 "name": SCENARIOS[key]["name"],
122 "n_runs": N_RUNS,
123 "violations": n_viol,
124 "ramp_breaches": n_ramp_bad,
125 "nan_runs": n_nan,
126 "worst_margin_psi": worst,
127 "final_Q_mean": float(finals.mean()),
128 "final_Q_std": float(finals.std()),
129 "final_Q_min": float(finals.min()),
130 "final_Q_max": float(finals.max()),
131 "target": runs[0]["final_target"],
132 "choke_std_tail_mean": float(np.mean([r["choke_std_tail"] for r in runs])),
133 "pass": ok,
134 "_runs": runs,
135 }
136 print(
137 f"[{key}] {N_RUNS} runs | violations={n_viol} ramp_breaches={n_ramp_bad} "

```

```

138 f"nan={n_nan} | worst margin={worst:.1f} psi | "
139 f"final Q={finals.mean():.1f}+/-{finals.std():.1f} bbl/hr -> "
140 f"'PASS' if ok else 'FAIL'"
141 )
142
143 # Production cost of the safety margins, measured against ground truth.
144 c = results["C"]
145 give_away = q_max_safe - c["final_Q_mean"]
146 print(
147 f"\nGround-truth max safe rate: {q_max_safe:.1f} bbl/hr at u={u_max_safe:.1f} %"
148 f"\nScenario C achieved: {c['final_Q_mean']:.1f} bbl/hr"
149 f"\nSafety margins cost: {give_away:.1f} bbl/hr "
150 f"({100 * give_away / q_max_safe:.1f} % of achievable rate)"
151 )
152
153 # ----- figure -----
154 fig, axes = plt.subplots(1, 3, figsize=(13, 3.8))
155 colors = {"WHP": COLOR_WHP, "FLP": COLOR_FLP, "BHP": COLOR_BHP}
156
157 for ax, key in zip(axes[:2], ("A", "C")):
158     runs = results[key]["_runs"]
159     for var in ("WHP", "FLP", "BHP"):
160         ax.hist([r["margins"][var] for r in runs], bins=18, alpha=0.65,
161                 color=colors[var], label=var)
162         ax.axvline(0, color="#dc2626", lw=1.8, ls="--", label="limit")
163     style_axis(ax, title=f"Scenario {key}: worst margin per run",
164               xlabel="Distance to nearest limit [psi]", ylabel="runs")
165     ax.legend(fontsize=7)
166
167 ax = axes[2]
168 for key, col in zip(("A", "B", "C"), (COLOR_Q, "#7c3aed", COLOR_BHP)):
169     finals = [r["final_Q"] for r in results[key]["_runs"]]
170     ax.hist(finals, bins=16, alpha=0.65, color=col, label=f"{key} (target {results[key]['target']:.0f})")
171     ax.axvline(q_max_safe, color="#111827", lw=1.6, ls=":", label=f"max safe {q_max_safe:.0f}")
172     style_axis(ax, title="Settled rate across 100 noise seeds",
173               xlabel="Final oil rate [bbl/hr]", ylabel="runs")
174     ax.legend(fontsize=7)
175
176 fig.suptitle(f"Monte-Carlo robustness: {N_RUNS} seeds x 3 scenarios", fontweight="bold")
177 fig.tight_layout()
178 savefig(fig, FIG_DIR / "robustness.png")
179 print(f"\n wrote {FIG_DIR / 'robustness.png'}")
180
181 for v in results.values():
182     v.pop("_runs")
183     results["_meta"] = {
184         "n_runs_per_scenario": N_RUNS,
185         "q_max_safe_ground_truth": q_max_safe,
186         "u_max_safe_ground_truth": u_max_safe,
187         "margin_cost_bblhr": give_away,
188         "all_pass": all_pass,
189     }
190 (DATA_DIR / "robustness_summary.json").write_text(json.dumps(results, indent=2))
191 print(f" wrote {DATA_DIR / 'robustness_summary.json'}")
192
193 print("\n" + "=" * 62)
194 print(f"ROBUSTNESS ({3 * N_RUNS} closed-loop runs):", "PASS" if all_pass else "FAIL")
195 print("=" * 62)
196 if not all_pass:
197     sys.exit(1)
198
199
200 if __name__ == "__main__":
201     main()
202

```